

# Design & Analysis of Algorithms

## Lab 1: Algorithm Performance Analysis Workbench

Name: Prachi Manwal

Roll No: 2501730365

Email: 2501730365@krmu.edu.in

---

### Q1. 1.2 Linear Search and Binary Search Comparison (5 marks)

```
def compare_search_algorithms(arr, target):
```

```
    #Linear search
```

```
    lin_index = -1
```

```
    lin_comps = 0
```

```
    for i in range(len(arr)):
```

```
        lin_comps += 1
```

```
        if arr[i] == target:
```

```
            lin_index = i
```

```
            break
```

```
    # Binary Search
```

```
    bin_index = -1
```

```
    bin_comps = 0
```

```
    low = 0
```

```
    high = len(arr) - 1
```

```
    while low <= high:
```

```
        bin_comps += 1
```

```
        mid = (low + high) // 2
```

```
        if arr[mid] == target:
```

```
            bin_index = mid
```

```
            high = mid - 1
```

```
        elif arr[mid] < target:
```

```
            low = mid + 1
```

```
        else:
```

```
            high = mid - 1
```

```
    # which algo performed better
```

```
    if bin_comps < lin_comps:
```

```
        better = "Binary Search"
```

```
    elif lin_comps < bin_comps:
```

```
        better = "Linear Search"
```

```
    else:
```

```
        better = "Both Equal"
```

```
    # Format as a list of strings
```

```
    return[
```

```
        "Search Comparison Report",
```

```
        "Linear Search",
```

```
        f"Index: {lin_index}",
```

```
        f"Comparisons: {lin_comps}",
```

```
        "Binary Search",
```

```
        f"Index: {bin_index}",
```

```
        f"Comparisons: {bin_comps}",
```

```
        f"Better Algorithm: {better}",
```

```
    ]
```

### Q2. 1.3 Bubble Sort and Insertion Sort Performance Comparison (10 marks)

```
def compare_bubble_insertion(random_data, sorted_data, reverse_data):
```

```
    datasets = [
```

```
        ("Random Dataset", list(random_data)),
```

```
        ("Sorted Dataset", list(sorted_data)),
```

```
        ("Reverse Dataset", list(reverse_data))
```

```
    ]
```

```
    report_lines = ["Sorting Performance Report"]
```

```
    for name, data in datasets:
```

```
        # Optimised bubble sort
```

```
        b_arr = list(data)
```

```
        b_comps = 0
```

```
        b_swaps = 0
```

```
        n = len(b_arr)
```

```
        for i in range(n - 1):
```

```
            swapped = False
```

```
            for j in range(n - 1 - i):
```

```
                b_comps += 1
```

```
                if b_arr[j] > b_arr[j + 1]:
```

```
                    b_arr[j], b_arr[j + 1] = b_arr[j + 1], b_arr[j]
```

```

        b_swaps += 1
        swapped = True
    if not swapped:
        break
# Insertion Sort
i_arr = list(data)
i_comps = 0
i_shifts = 0

for i in range(1, len(i_arr)):
    key = i_arr[i]
    j = i - 1
    while j >= 0:
        i_comps += 1
        if i_arr[j] > key:
            i_arr[j + 1] = i_arr[j]
            i_shifts += 1
            j -= 1
        else:
            break
    i_arr[j + 1] = key

# Determine better algo
if b_comps < i_comps:
    better = "Bubble Sort"
elif i_comps < b_comps:
    better = "Insertion Sort"
else:
    better = "Both Equal"

# Append lines for this dataset
report_lines.append(name)
report_lines.append(f"Bubble Sorted: {' '.join(map(str, b_arr))}")
report_lines.append(f"Bubble Comparisons: {b_comps}")
report_lines.append(f"Bubble Swaps: {b_swaps}")
report_lines.append(f"Insertion Sorted: {' '.join(map(str, i_arr))}")
report_lines.append(f"Insertion Comparisons: {i_comps}")
report_lines.append(f"Insertion Shifts: {i_shifts}")
report_lines.append(f"Better Algorithm: {better}")

return report_lines

```